

Projeto: Agente Autônomo de Infraestrutura com Governança

Visão do Projeto

Criar um agente de IA que execute tarefas de infraestrutura de forma segura, auditável e governada, permitindo comandos em linguagem natural para provisionamento, configuração, operação e correção automatizada — mantendo controle humano, conformidade e rastreabilidade.

Problema que resolve

- Mudanças manuais e lentas em infraestrutura
 - Erros por execução sem padrão
 - Falta de governança em automação tradicional
 - Incidentes por falha humana
 - Falta de visibilidade do estado real da infra
-

Objetivo Principal

Permitir que alguém diga, por exemplo: "Provisione um Wazuh cluster com manager no servidor 1, indexar no 2 e replicação nos servidores 3 e 4"

E o agente: - entende o pedido - valida políticas e riscos - gera plano - executa com segurança - monitora - trata erros - documenta automaticamente

Arquitetura Macro

Interface Web → Planner IA → Validador / Governança → Executor → Observabilidade / Self-Healing

Componentes Principais

1) Interface Web (Portal do Agente)

- Chat / formulário de requisições
- RBAC (quem pode pedir o quê)
- Histórico e auditoria
- Aprovação humana

2) Cérebro IA (Planner)

- Interpreta comandos naturais
- Cria plano técnico

- Gera Terraform / Ansible / Helm
- Define dependências e ordem de execução

3) Governança e Segurança

- Policies (OPA / Checkov)
- Compliance
- Limites de custo
- Aprovação obrigatória para mudanças críticas

4) Executor Controlado

- GitOps (ArgoCD / Flux) ou CI/CD
- Sem root livre
- Acesso com menor privilégio

5) Observabilidade e Erros

- Health-checks automáticos
- Logs estruturados
- Retry inteligente
- Rollback seguro
- Notificação Slack/Teams/Email

Fonte de Verdade da Infra (State / CMDB)

Para o agente realmente "saber" sobre a infraestrutura: - Inventário (NetBox / Ansible Inventory) - Terraform State - API Cloud (AWS/GCP/Azure) ou Proxmox/VMware - CMDB - Métricas e Logs

Fluxo Padrão de Operação

1) Usuário solicita 2) IA entende e monta plano 3) Verificação de políticas e segurança 4) Geração de artefatos (Terraform / Ansible / Helm) 5) Pull Request → Aprovação humana 6) Execução segura / GitOps 7) Monitoramento ativo 8) Correção automática se possível 9) Rollback se necessário 10) Documentação automática

Tecnologias Recomendadas

Frontend: Next.js / React
Backend: FastAPI ou Node.js
IA Planner: OpenAI / Claude + Regras + RAG
Infra: Terraform / Pulumi
Configuração: Ansible / Helm
Orquestração: Temporal / Airflow / CI
GitOps: ArgoCD / Flux
Governança: OPA, Checkov, Conftest
Observabilidade: Prometheus, Grafana, Loki, ELK

Tratamento de Erros (Self-Healing)

- Detectar falhas
- Retry automático limitado
- Correção de erros conhecidos
- Recriar componentes isolados
- Rollback seguro
- Notificar humano se impacto crítico

Regras de Governança

- Nada executa diretamente sem validação
- Mudanças críticas exigem aprovação
- Sem credenciais root
- Auditoria completa
- Logs obrigatórios

Roadmap de Implementação

Fase 1 – MVP

- Interface básica
- Planner IA simples
- Executor via pipeline
- Tratamento básico de erro

Fase 2 – Governança Avançada

- Políticas e compliance
- RBAC completo
- Observabilidade integrada

Fase 3 – Autonomia Controlada

- Self-healing completo
- Automação complexa multi-servidor
- Catálogo de operações prontas

Riscos

- IA executar algo fora de contexto → limitar ações
 - Falhas sem rollback → orquestração com estado
 - Acesso excessivo → privilégio mínimo
-

Métricas de Sucesso

- Redução do tempo médio de implantação
 - Redução de erros humanos
 - Redução de incidentes
 - Taxa de aprovações x rejeições de mudanças
-

Cenários Suportados

- Web + Banco de Dados + Replica
 - Wazuh Cluster
 - Deploy Kubernetes
 - Infra cloud e on-prem
-

Próximos Passos

- Escolher stack final
 - Definir fonte de verdade
 - Implementar MVP
 - Evoluir para governança total
-

Se quiser, posso transformar este canvas em: - diagrama arquitetural - backlog JIRA - documentação técnica - blueprint Terraform/Ansible